

Componentes y Modelos de Componentes

ÍNDICE

Marco Teórico: Componentes (Ian Sommerville)	1
Marco Teórico: Modelos de Componentes (Ian Sommerville)	2
Un ejemplo didáctico para entender cómo crear una muestra de modelo de componentes desde cero en C++	8
¿Por qué esta arquitectura es robusta?	9
Implementación de con ejemplo de componente Saludador (Greeter)	11
i_component.hpp:	11
i_greeter.hpp:	13
shared_library.hpp:	14
module_manager.hpp:	17
Gestión Segura del Ciclo de Vida Cruzado (Custom Deleter & RAII)	22
greeter_component.cpp:	23

Marco Teórico: Componentes (Ian Sommerville)

Un componente puede ser definido como un elemento de software que se conforma a un modelo de componentes estándar y puede desplegarse y componerse independientemente sin modificación, de acuerdo con un estándar de composición.

También un componente de software es una unidad de composición con interfaces especificadas contractualmente y sólo con dependencias de contexto explícitas. Un componente de software puede implementarse de manera independiente y puede estar sujeto a composición por terceras partes.

Característica del componente	Descripción
Estandarizado	Estandarización de componentes significa que un componente utilizado durante un proceso CBSE debe ajustarse a un modelo de componentes estándar. Este modelo puede definir interfaces de componentes, metadatos de componentes, documentación, composición e implementación.
Independiente	Un componente debe ser independiente; debe ser factible componerlo e implementarlo sin usar otros componentes específicos. En situaciones en que el componente necesita brindar servicios externos, esto debería plantearse claramente en una especificación de interfaz de "requiere".
Componible	Para que un componente sea componible, todas las interacciones externas deben tener lugar mediante interfaces definidas públicamente. Además, debe permitir acceso externo a información acerca de sí mismo, así como a sus métodos y atributos.
Implementable	Para que sea implementable, un componente debe estar autocontenido. Debe ser capaz de ejecutarse como entidad independiente en una plataforma de componente que permita una implementación del modelo de componentes. Por lo general, esto significa que el componente es binario y no tiene que compilarse antes de su implementación. Si un componente se implementa como servicio, no tiene que implementarse por parte de un usuario de un componente. En vez de ello, se implementa por parte del proveedor del servicio.
Documentado	Los componentes deben implementarse por completo, para que los usuarios potenciales puedan decidir si los componentes cumplen o no sus necesidades. Debe especificarse la sintaxis y, de manera ideal, la semántica de todas las interfaces de componente.

Visualizar un componente como un proveedor de servicio pone de relieve dos características críticas de un componente de reutilización:

1. El componente es una entidad ejecutable independiente definida mediante sus interfaces. *Para usarlo no se necesita conocimiento alguno de su código fuente.* Puede hacerse referencia a él como un servicio externo o incluirse directamente en un programa.
2. Los servicios ofrecidos por un componente se ponen a disposición mediante una interfaz, y todas las interacciones por dicha interfaz. *La interfaz del componente se expresa en términos de operaciones parametrizadas y nunca se expone su estado interno.*

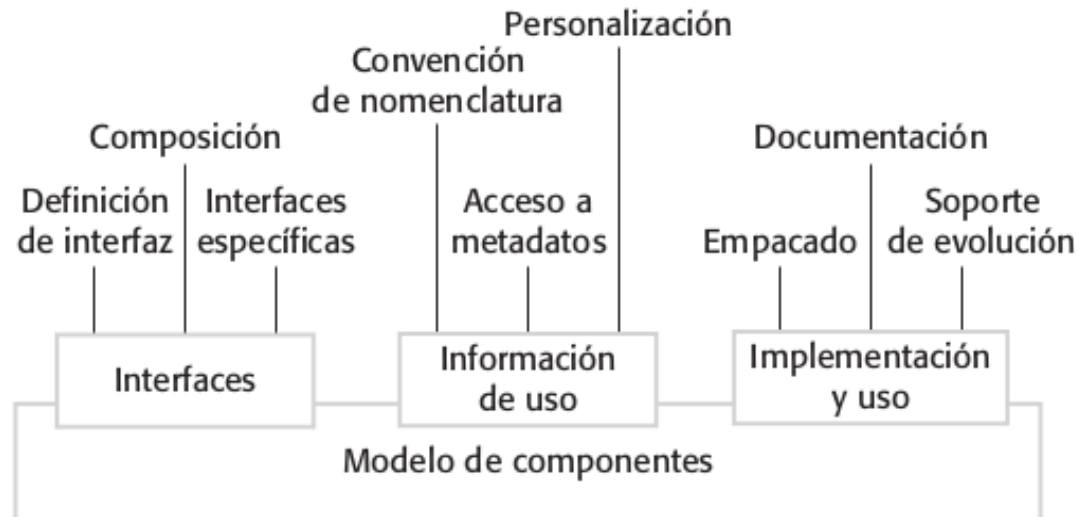
Marco Teórico: Modelos de Componentes (Ian Sommerville)

Un modelo de componentes es una definición de estándares para implementación, documentación y despliegue de componentes. Estos estándares se establecen con la finalidad de que los desarrolladores de componentes se aseguren de que éstos pueden interoperar.

También funcionan para proveedores de infraestructuras de ejecución de componentes que ofrecen middleware para apoyar la ejecución de componentes. Se han propuesto muchos modelos de componentes, pero ahora los modelos más importantes son el modelo WebServices, el modelo Enterprise Java Beans (EJB) de Sun, y el modelo .NET de Microsoft (Lau y Wang, 2007). También podemos agregar actualmente el modelo de componentes de WebAssembly (Wasm Component Model).

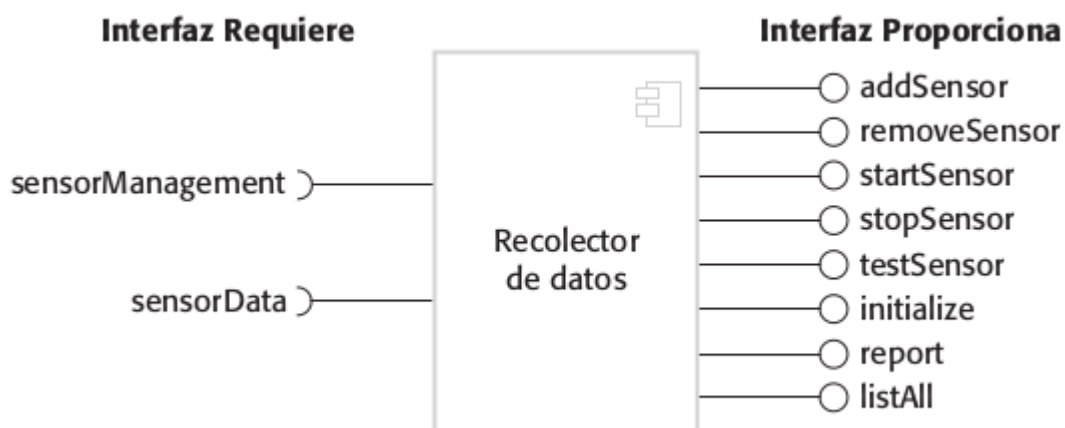
Weinreich y Sametinger (2001) analizan los elementos básicos de un modelo ideal de componentes. Este diagrama muestra que los elementos de un modelo de componentes definen las interfaces de

componentes, la información que necesita usar el componente en un programa y cómo debe implementarse un componente:



- 1. Interfaces:** Los componentes se definen al especificar sus interfaces. El modelo de componentes especifica cómo deben definirse las interfaces y los elementos, tales como los nombres de operación, los parámetros y las excepciones que deben incluirse en la definición de la interfaz. El modelo también debe especificar el lenguaje usado para definir las interfaces de componentes. Algunos modelos de componentes requieren interfaces específicas que deben definirse por un componente. Se usan para componer el componente con la infraestructura de modelo de componentes, que ofrece servicios estandarizados, tales como seguridad y gestión de transacción.
- 2. Información de Uso:** Para que los componentes se distribuyan y se acceda a ellos de manera remota, deben tener un nombre único asociado. Éste debe ser totalmente único, por ejemplo, en EJB (Enterprise Java Beans) se genera un nombre jerárquico con la raíz basada en un nombre de dominio de Internet. Los servicios tienen un URI único (Uniform Resource Identifier, esto es, un

identificador de recursos uniforme). Los metadatos de componente son datos acerca del componente en sí, tales como información acerca de sus interfaces y atributos. Los metadatos son importantes porque permiten a los usuarios del componente determinar qué servicios se proporcionan y requieren. Las implementaciones de modelos de componentes por lo general incluyen formas específicas (tales como el uso de una interfaz de reflexión en Java) para acceder a los metadatos de este componente. Los componentes son entidades genéricas y, cuando se implementan, deben configurarse para ajustarse en un sistema de aplicación. Por ejemplo, se podría configurar el componente recolector de datos al definir el número máximo de sensores en un arreglo.



Por lo tanto, el modelo de componentes puede especificar cómo pueden personalizarse los componentes binarios para un entorno de implementación particular.

3. Implementación: El modelo de componentes incluye una especificación de cómo deben empacarse los componentes para su implementación como entidades ejecutables independientes. Puesto que los componentes son entidades independientes, deben empacarse con todo el software de soporte que no proporcione la infraestructura de componente, o que no esté definido en una interfaz

"requiere". La información de implementación incluye información sobre el contenido de un paquete y su organización binaria. Inevitablemente, conforme surjan nuevos requerimientos, los componentes deberán cambiarse o sustituirse. Por consiguiente, el modelo de componentes puede incluir reglas que rigen cuándo y cómo se permite la sustitución de componentes. Finalmente, el modelo de componentes puede definir la documentación de componentes que deba producirse. Esto se usa para encontrar el componente y decidir si es adecuado.

Algunos ejemplos de modelos de componentes que han sido utilizados en industria de software con mayor impacto histórico son:

1. El Modelo de Componentes CORBA (CCM - CORBA Component Model)

<https://www.omg.org/corba/> es una extensión del modelo de objetos distribuidos de CORBA. Define un marco estandarizado para implementar, configurar, desplegar y ensamblar componentes de software que funcionan en redes heterogéneas, independientemente del lenguaje de programación o sistema operativo.

Arquitectura y Componentes Principales

El CCM transforma la programación de objetos distribuidos en un enfoque de ensamblaje de componentes (COTS - Components "Off-The-Shelf"). Su arquitectura se basa en las siguientes características:

- Puertos de Interacción (Ports): Son los puntos de contacto del componente con el exterior, divididos en:
 - Facets (Facetas): Interfaces que el componente ofrece al exterior (provee servicios).

- **Receptacles (Receptáculos):** Representan dependencias; son los puntos donde el componente requiere servicios de otros.
- **Event Sinks:** Canales de entrada para recibir eventos asíncronos.
- **Event Sources:** Canales de salida para emitir eventos a otros componentes.
- **Contenedor (Container):** Entorno de ejecución que aloja al componente y gestiona servicios comunes de manera transparente (ej. transacciones, seguridad, persistencia).
- **Home:** Factorías de componentes. Es la entidad encargada de crear, destruir, buscar y gestionar el ciclo de vida de los componentes dentro del contenedor.
- **Despliegue e Implementación (Deployment):** El CCM estandariza metadatos (a través de descriptores XML) que permiten a los administradores ensamblar aplicaciones complejas combinando componentes previamente construidos.

2. COM (Component Object Model) / DCOM (Microsoft):

- **Estado:** Vigente y fundamental en la arquitectura de Windows (DirectX, OLE, Windows Runtime).
- **Funcionamiento:** Resuelve el problema del ABI (Application Binary Interface) en C++ utilizando tablas de funciones virtuales (vtables) puras, las cuales son binariamente compatibles con estructuras de C. Exige el uso de interfaces puramente abstractas y un sistema de recuento de referencias (métodos **AddRef** y **Release**) heredados de una interfaz base **IUnknown**.
- **Relevancia:** Es el caso de estudio por excelencia sobre cómo construir componentes agnósticos del compilador en C/C++.

3. OSGi (Open Services Gateway initiative):

- **Estado:** Muy utilizado en el ecosistema Java (ej. el IDE Eclipse, servidores de aplicaciones).
- **Funcionamiento:** Proporciona un modelo dinámico para Java donde los componentes (llamados *bundles*) pueden ser instalados, iniciados, detenidos, actualizados y desinstalados sin reiniciar la máquina virtual. Gestiona estrictamente las dependencias y el control de versiones a nivel de paquetes.

4. WebAssembly (Wasm) Component Model:

- **Estado:** En desarrollo activo (respaldado por la Bytecode Alliance y el W3C), emergiendo como el estándar moderno para componentes.
- **Funcionamiento:** Permite que módulos escritos en diferentes lenguajes (C++, Rust, Go, Python) interactúen de forma segura y agnóstica a la arquitectura subyacente. Define tipos de datos compartidos de alto nivel (cadenas, registros) sin depender de la memoria lineal del módulo subyacente.

Un ejemplo didáctico para entender cómo crear una muestra de modelo de componentes desde cero en C++

<https://github.com/gabrielinuz/cpp-component-model>

Para construir un modelo de componentes multiplataforma y seguro en C++, la filosofía central debe ser **"El que reserva la memoria, la libera"** (para evitar la corrupción del heap) y **"El contrato es puro"** (para evitar problemas de **ABI**¹).

Para lograr esto, aplicaremos los siguientes patrones modernos:

- **Interfaces Puras (ABI-Safe):** Las clases solo tendrán funciones virtuales puras. No cruzaremos el límite del módulo con tipos complejos de la biblioteca estándar (`std::string`, `std::vector`), usaremos tipos primitivos **C-style**.
- **Fábricas C-Linkage (`extern "C"`):** Cada módulo exportará una función para crear la instancia y otra para destruirla.
- **RAII² para las Bibliotecas Dinámicas:** Envolveremos los *handles* del sistema operativo (`void*` de `dlopen`) en una clase C++ que garantice su cierre.
- **Deleters Personalizados (La magia real):** Usaremos `std::shared_ptr`, pero le inyectaremos un *custom deleter* (eliminador personalizado). Este eliminador llamará a la función de destrucción de la biblioteca y, crucialmente, mantendrá vivo el *handle* de la biblioteca hasta que el último componente que la use haya sido destruido.

¹ El término ABI (Application Binary Interface o Interfaz Binaria de Aplicación) define el contrato de bajo nivel entre dos módulos de software compilados o un componente y el sistema operativo. En el modelo de componentes, la ABI establece exactamente cómo interactúan a nivel de código máquina y de memoria.

² RAI (por sus siglas en inglés: Resource Acquisition Is Initialization o "la adquisición de recursos es la inicialización") es un patrón de diseño fundamental en C++ y Rust creado por Bjarne Stroustrup. Garantiza que los recursos (memoria, archivos, sockets) se asignen durante la creación del objeto y se liberen automáticamente al salir de su ámbito, evitando fugas.

¿Por qué esta arquitectura es robusta?

1. **Fin del "Heap Corruption":** No se debe usar `std::make_shared` dentro del módulo. Cuando ese puntero expira en `main.cpp`, el binario principal intenta liberar memoria que no le pertenece. Con el **Deleter Personalizado** (`[destroy_func](InterfaceType* ptr)`), el `main.cpp` le ordena a la biblioteca compartida³ (shared library): *"Tú creaste este puntero, libéralo tú"*.
2. **Protección contra la Descarga Prematura (Dangling Libraries):** Al capturar la variable `lib` (que es el `std::shared_ptr<SharedLibrary>`) dentro de la lambda del deleter, obligas a C++ a incrementar el contador de referencias de la biblioteca completa. Es físicamente imposible que la biblioteca se cierre (ejecute `dlclose`) mientras exista al menos un componente utilizándola, eliminando los *segfaults* por acceder a memoria de un módulo descargado.
3. **Mayor robustez con ABI:** Al cambiar `std::string` en la firma de las funciones por `const char*` y buffers mutables, se consigue mayor robustez ante el *name mangling* y a las diferencias entre versiones de la Standard Template Library. Un componente compilado hoy funcionará en un ejecutable compilado dentro de 10 años.
4. **Seguridad de Hilos y Concurrencia Primitiva:** El mapa interno de bibliotecas cargadas (`loaded_libraries`) dentro del **ModuleManager** se encuentra protegido de accesos concurrentes asíncronos mediante exclusión mutua con `std::mutex` y bloqueos

³ Es un archivo de código compilado que varios programas pueden utilizar al mismo tiempo sin necesidad de duplicarlo en el equipo. Por ejemplo, en Linux se conocen como archivos `.so` y en Windows como `.dll`.

estructurados de tipo **std::lock_guard**. Esto garantiza la estabilidad del sistema si múltiples hilos del Host intentan instanciar o cargar componentes de forma simultánea.

5. Rechazo a std::exit en Infraestructura (Propagación vía

Excepciones): Se rechazó el uso de cortes abruptos mediante **std::exit(EXIT_FAILURE)** dentro del mánager de módulos. Invocar **std::exit** aborta el programa omitiendo el desenredo de la pila (**stack unwinding**), impidiendo la ejecución de destructores locales activos y dejando descriptores o recursos abiertos. En su lugar, los fallos de infraestructura se elevan limpiamente al Host mediante **std::runtime_error**, permitiendo una finalización ordenada y segura.

6. Control de Versiones Estricto: Si se compila el Host con una versión de **IComponent** que tiene 4 métodos, y cargas una biblioteca compartida (.dll o .so) compilada con una versión antigua que solo tiene 3 métodos, el Host intentará leer una tabla de funciones virtuales (**vtable**) incorrecta, provocando una violación de segmento (**Segfault**). **Solución obligatoria para producción:** El módulo exporta una función que devuelva su versión del **ABI** (ej. **extern "C" int get_api_version()**) para que el Host la valide antes de instanciar nada.

Implementación de con ejemplo de componente Saludador (Greeter)

i_component.hpp:

Define el contrato del ciclo de vida base del componente, la versión del ABI (**CURRENT_API_VERSION**) y los tipos de punteros a función de la C-API.

```
#ifndef ICOMPONENT_HPP
#define ICOMPONENT_HPP

enum class ComponentResult : int
{
    SUCCESS = 0,
    ERROR_INVALID_ARGUMENT = 1,
    ERROR_INTERNAL = 2
};

constexpr int CURRENT_API_VERSION = 1;

class IComponent
{
public:
    virtual ~IComponent() noexcept = default;
};

extern "C"
{
    typedef int (*get_api_version_func)() noexcept;

    typedef IComponent* (*create_component_func)() noexcept;

    typedef void (*destroy_component_func)(IComponent*) noexcept;
}

#endif // ICOMPONENT_HPP
```

Utilizamos un enum class (En C++, se utiliza para agrupar constantes lógicas de manera segura.) para códigos de estado de las operaciones a través del ABI. Al evitar excepciones, requerimos un mecanismo clásico de retorno de errores (tipo HRESULT en COM) para informar al Host sobre el estado:

```
enum class ComponentResult : int
{
    SUCCESS = 0,
    ERROR_INVALID_ARGUMENT = 1,
    ERROR_INTERNAL = 2
};
```

Versión actual del contrato ABI. Si se modifican las interfaces virtuales (agregando o quitando métodos), esta expresión constante DEBE incrementarse para evitar violaciones de segmento:

```
constexpr int CURRENT_API_VERSION = 1;
```

Interfaz base para todos los componentes, solo un Destructor virtual. El uso de 'noexcept' es obligatorio aquí. Evita que una excepción lanzada durante la destrucción intente cruzar el límite del módulo:

```
class IComponent
{
public:
    virtual ~IComponent() noexcept = default;
};
```

Tipos de punteros a función exportados (C-API) esto permite portabilidad entre sistemas operativos y tipos de compiladores ya que los símbolos en C permanecen constantes a diferencia de C++. Todas las funciones que cruzan el límite de la biblioteca compartida están marcadas con 'nothrow' garantizando que el compilador aborte el programa internamente si ocurre un error no manejado, en lugar de corromper la pila del Host.

```
extern "C"
{
    typedef int (*get_api_version_func)() noexcept;

    typedef IComponent* (*create_component_func)() noexcept;

    typedef void (*destroy_component_func)(IComponent*) noexcept;
}
```

i_greeter.hpp:

Interfaz específica para el componente Saludador:

```
#ifndef IGREETER_HPP
#define IGREETER_HPP

#include "IComponent.hpp"
#include <cstdint>

class IGreeter : public IComponent
{
public:
    virtual ComponentResult greet(const char* name, char* out_buffer,
                                   size_t buffer_size) noexcept = 0;
};

#endif // IGREETER_HPP
```

shared_library.hpp:

Clase RAII para gestionar el ciclo de vida de una biblioteca compartida: https://github.com/gabrielinuz/cpp-component-model/blob/main/include/shared_library.hpp

Con directivas del preprocesador se fuerza para compilar condicionalmente código específicamente para cada sistemas operativos, esto permite detectar la plataforma y cargar la biblioteca adecuada:

```
#ifndef __unix__

#include <dlfcn.h>

const std::string LIB_EXTENSION = ".so";

#elif defined(_WIN32) || defined(WIN32)

#include <windows.h>

const std::string LIB_EXTENSION = ".dll";

#elif __APPLE__

#include <dlfcn.h>

const std::string LIB_EXTENSION = ".dylib";

#else

#error "Plataforma no soportada"

#endif
```

Clase RAII para gestionar el ciclo de vida de una biblioteca compartida:

```
class SharedLibrary
{
private:
    void* handle;

    std::string path;
```

Constructor que intenta cargar la biblioteca, el parámetro **libPath** es Ruta de la biblioteca (sin la extensión del SO). Se lanza un **std::runtime_error** Si la biblioteca no se puede cargar. En C++, La palabra reservada **explicit** impide que el compilador utilice un constructor para conversiones de tipo implícitas. Obliga al código del desarrollador a inicializar los objetos intencionalmente mediante inicialización directa o conversión explícita.

```
public:
    explicit SharedLibrary(const std::string& libPath) : path(libPath +
LIB_EXTENSION)
    {
        #ifdef _WIN32
            handle = LoadLibrary(path.c_str());
        #else
            handle = dlopen(path.c_str(), RTLD_NOW | RTLD_LOCAL);
        #endif
        if (!handle)
        {
            throw std::runtime_error("Error al cargar la biblioteca:
                                     " + path);
        }
    }
}
```


Destructor que libera la biblioteca de la memoria, utilizando directivas del preprocesador para hacerlo adecuadamente en el sistema operativo donde se compila.

```
~SharedLibrary()
{
    if (handle)
    {
        #ifdef _WIN32
            FreeLibrary((HINSTANCE)handle);
        #else
            dlclose(handle);
        #endif
    }
}
```

Prevenimos la copia para evitar cerrar el handle accidentalmente agregando una línea **= delete;** a la declaración de un constructor. Esto indica al compilador que desactive dicho constructor, impidiendo que los objetos se creen o copien de esa manera específica:

```
SharedLibrary(const SharedLibrary&) = delete;

SharedLibrary& operator=(const SharedLibrary&) = delete;
```

El método **get_symbol** obtiene un símbolo (función) exportado por la biblioteca. El parámetro **symbolName** es el nombre de la función exportada. Este método retorna un puntero a la función, o `nullptr` si no se encuentra.

```
void* get_symbol(const std::string& symbolName)
{
    #ifdef _WIN32
        return (void*)GetProcAddress((HINSTANCE)handle,
                                      symbolName.c_str());
    #else
        return dlsym(handle, symbolName.c_str());
    #endif
};
#endif // SHARED_LIBRARY_HPP
```

module_manager.hpp:

https://github.com/gabrielinuz/cpp-component-model/blob/main/include/module_manager.hpp

Gestor central de módulos que resuelve la instanciación segura. Mantiene una frontera estricta de infraestructura. No incorpora lógica de negocio ni conoce los métodos específicos de las interfaces derivadas. Hace uso intensivo de `shared_library.hpp`. Y se usa un contenedor `map` (mapa) para guardar las distintas bibliotecas dinámicas que se carguen y se manipulen como módulos:

```
class ModuleManager{
    private:
        std::unordered_map<std::string, std::shared_ptr<SharedLibrary>>
loaded_libraries;
```

Hacemos uso de **mutex** con **std::lock_guard**. Un **mutex** (exclusión mutua) en C++ es un mecanismo de sincronización que permite proteger recursos compartidos para evitar que múltiples hilos accedan a ellos al mismo tiempo. Garantiza que solo un hilo a la vez ejecute la sección crítica de código. En este caso lo utilizamos para proteger el acceso concurrente al mapa de bibliotecas.

```
std::mutex map_mutex;
```

El método privado **extract_module_name**, lo usamos para extraer el nombre de una biblioteca desde una ruta completa, por ejemplo: Extrae **"Greeter"** a partir de **"./lib/Greeter"**.

```
std::string extract_module_name(const std::string& path) const
{
    size_t pos = path.find_last_of("/\\");
    return (pos == std::string::npos) ? path : path.substr(pos + 1);
}
```

El método público **load_module** carga un módulo en memoria usando la ruta proporcionada. El parámetro **path** es ruta real del archivo (sin extensión). Mediante el manejo de excepciones se lanza **std::runtime_error** si el módulo no puede ser cargado, deteniendo la ejecución en cascada de dependencias mediante RAII.

```

void load_module(const std::string& path)
{
    std::string module_name = extract_module_name(path);

    try
    {
        auto lib = std::make_shared<SharedLibrary>(path);

        // Bloqueo del mutex antes de modificar el mapa compartido

        std::lock_guard<std::mutex> lock(map_mutex);

        loaded_libraries[module_name] = lib;

    }

    catch (const std::exception& e)
    {
        /* En lugar de retornar false o usar exit(),
           lanzamos una excepción controlada del Host */

        throw std::runtime_error("ModuleManager Error Crítico
al cargar " + module_name + ": " + e.what());

    }

}

```

El método público **create_instance** está parametrizado mediante el uso de template para recibir la interfaz/tipo a la cual debe ser casteado el componente. Este método crea una instancia validando la versión del ABI⁴. El parámetro **module_name** es el nombre del archivo extraído de la ruta (ej. "Greeter"). Si se atrapa alguna excepción se lanza **std::runtime_error** si hay

⁴ El término ABI (Application Binary Interface o Interfaz Binaria de Aplicación) define el contrato de bajo nivel entre dos módulos de software compilados o un componente y el sistema operativo. En el modelo de componentes, la ABI establece exactamente cómo interactúan a nivel de código máquina y de memoria.

discrepancias o errores internos; garantizando que el Host no reciba jamás un puntero inválido o nullptr.

Es importante agregar que se crea un **std::shared_ptr** con un **custom deleter** (eliminador personalizado), este eliminador se pasa como segundo argumento en su constructor. Este puede ser una función normal, un functor (clase con el operador () sobrecargado), o una expresión lambda.

Este método es un candidato futuro a revisar bajo el principio de responsabilidad única. A continuación se copia su código con comentarios aclaratorios:

```
template<typename InterfaceType>
    std::shared_ptr<InterfaceType> create_instance(const std::string&
module_name)
    {
        std::shared_ptr<SharedLibrary> lib;
```

*Ámbito artificial para reducir el tiempo de bloqueo del **mutex**:*

```
{
    std::lock_guard<std::mutex> lock(map_mutex);
    auto it = loaded_libraries.find(module_name);
    if (it == loaded_libraries.end())
    {
        throw std::runtime_error("ModuleManager Error: Módulo
requerido no precargado -> " + module_name);
    }
    lib = it->second;
}
```

Obtención de las funciones externas como punteros a función:

```
auto get_api_func = (get_api_version_func)lib->get_symbol("get_api_version");
auto create_func = (create_component_func)lib->get_symbol("create_component");
auto destroy_func =
(destroy_component_func)lib->get_symbol("destroy_component");

if (!get_api_func || !create_func || !destroy_func){

    throw std::runtime_error("ModuleManager Error: Faltan símbolos
requeridos en " + module_name);

}
```

Control de versiones estricto. Comparamos la versión de la interfaz con la que compiló la biblioteca compartida. contra la versión actual que maneja el Host. Si difieren, abortamos por excepción.

```
int module_api_version = get_api_func();

if (module_api_version != CURRENT_API_VERSION){

    throw std::runtime_error("ModuleManager Error: Discrepancia de ABI en " +
module_name + ". Esperado: " + std::to_string(CURRENT_API_VERSION) +
", Encontrado: " + std::to_string(module_api_version));

}
```

*Invocamos a la **biblioteca compartida** mediante su función creadora para crear el objeto en su propio **heap**.*

```
IComponent* raw_instance = create_func();

if (!raw_instance)

{

    throw std::runtime_error("ModuleManager Error: La fábrica de la biblioteca
                                devolvió una instancia nula.");

}
```

"Casteamos" a la **interfaz solicitada** luego si este casteo falla (entrega un **puntero nulo**) y limpiamos para evitar fugas invocando a la **función de destrucción** del componente:

```
InterfaceType* casted_instance=dynamic_cast<InterfaceType*>(raw_instance);
if (!casted_instance)
{
    destroy_func(raw_instance);

    throw std::runtime_error("ModuleManager Error: El componente " +
                             module_name + " no implementa la interfaz
solicitada.");
}
```

Gestión Segura del Ciclo de Vida Cruzado (Custom Deleter & RAII)

"LA MAGIA": Garantía de Coherencia de Memoria y Frontera Binaria. Este bloque resuelve dos de los problemas más críticos en arquitecturas de plugins:

1. Simetría en la Asignación (Asymmetry Mitigation):

- a. En C++, la memoria **DEBE** ser liberada en el mismo contexto (heap/runtime) en el que fue reservada. Delega la destrucción a **'destroy_func'** (dentro de la **biblioteca compartida**), evitando que la **aplicación host** intente hacer un **'delete'** local, lo que provocaría corrupción del heap o un crash inmediato.

2. Prevención de Descarga Prematura (Lifetime Interlocking via RAII):

- a. Al capturar el **'std::shared_ptr<SharedLibrary> lib'** por **VALOR dentro de la lambda**, incrementamos el contador de **referencias de la biblioteca compartida**. Esto genera un cerrojo lógico: incluso si el **'ModuleManager'** decide liberar la biblioteca o es destruido, el binario físico (**.so/.dll**) **PERMANECERÁ** mapeado en el espacio de direccionamiento del Host. La llamada del sistema (**'dlclose'**) solo ocurrirá cuando el último componente creado por este manager sea destruido y la lambda suelte su copia de **'lib'**. Esto es lo que verdaderamente

previene los **punteros colgantes (dangling pointers)** a nivel de biblioteca.

```
auto deleter = [destroy_func, lib](InterfaceType* ptr)
{
    destroy_func(ptr);
};

return std::shared_ptr<InterfaceType>(casted_instance, deleter);
}

};

#endif // MODULE_MANAGER_HPP
```

greeter_component.cpp:

Esta es la implementación concreta de componente ejemplo que respeta el límite sin excepciones. Para no propagarlas por fuera del ámbito de la biblioteca compartida.

https://github.com/gabrielinuz/cpp-component-model/blob/main/src/greeter_component.cpp

```
class GreeterComponent : public IGreeter
{
private:
    std::string prefix;

public:
    GreeterComponent() : prefix("Hello, ") {}

    ~GreeterComponent() noexcept override = default;
```

*Implementación envuelta en try-catch. Atrapa **cualquier excepción** C++ (como **std::bad_alloc** en caso de falta de memoria al concatenar **std::string**) impidiendo que cruce el ABI:*


```

ComponentResult greet(const char* name, char* out_buffer, size_t
buffer_size) noexcept override
{
    if (!name || !out_buffer || buffer_size == 0)
    {
        return ComponentResult::ERROR_INVALID_ARGUMENT;
    }
    try
    {
        std::string result = prefix + name + "!";

        if (result.length() >= buffer_size)
        {
            return ComponentResult::ERROR_INVALID_ARGUMENT;
        }

        strncpy(out_buffer, result.c_str(), buffer_size - 1);
        out_buffer[buffer_size - 1] = '\0';

        return ComponentResult::SUCCESS;
    }
}

```

Un bloque catch-all asegura que **NINGUNA** (operador triple punto ...) excepción escape hacia la aplicación host:

```

    catch (...)
    {
        return ComponentResult::ERROR_INTERNAL;
    }
}
};

```

La función externa `get_api_version` expone la versión con la que fue compilada esta biblioteca compartida:

```
extern "C" int get_api_version() noexcept
{
    return CURRENT_API_VERSION;
}
```

La función externa `create_component` retorna la instancia del **GreeterComponent** y si algo sale mal atrapa cualquier excepción retornando un puntero nulo:

```
extern "C" IComponent* create_component() noexcept
{
    try
    {
        return new GreeterComponent();
    }
    catch (...)
    {
        return nullptr;
    }
}
```

Por último la función externa **`destroy_component`** destruye la instancia que se le pasa por parámetro (utilizada en la **aplicación host** y creada por **`module_manager.hpp`**) en el contexto de memoria de la biblioteca compartida. Además el operador `delete` en C++ ya maneja internamente la comprobación de `nullptr`:

```
extern "C" void destroy_component(IComponent* instance) noexcept
{
    delete instance;
}
```